System Design Document (SDD)

Part 1: Database Architecture & Data Models

1. Database Strategy

- **Database Management System:** MongoDB (NoSQL).
- **ODM Library:** Mongoose (for Node.js).
- **Rationale:** MongoDB is selected for its flexibility with product specifications (a Laptop has different specs than a Phone) and its speed in handling geospatial queries (for future location features).

2. Entity Relationships (ER Logic)

The system relies on relational data stored in a document-based structure.

- **User 1:N Products:** One user (Seller) can list multiple products.
- **User 1:N Orders:** One user can have multiple buy/sell orders.
- **Order 1:1 Product:** For the MVP, an order strictly links a specific Buyer, a specific Seller, and a specific Product.
- **User 1:N Transactions:** Every wallet movement creates a transaction record for audit.

3. Schema Definitions (Mongoose Models)

3.1 User Entity (models/User.js)

Stores authentication data, profile details, and the internal wallet balance.

JavaScript

```javascript
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  // Identity & Auth
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, trim: true },
  password: { type: String, required: true }, // Hashed (bcrypt)
  phone: { type: String, required: true, unique: true }, // Verified via OTP

  // Location (Targeting Addis Ababa Sub-cities)
  address: {
    city: { type: String, default: 'Addis Ababa' },
    subCity: { type: String, required: false }, // e.g., 'Bole', 'Yeka'
    specificAddress: { type: String, default: "" }
  },

  // Role Management
  role: {
    type: String,
    default: "user",
    enum: ["user", "admin", "shop_owner"]
  },

  // Trust & Verification System
  isVerified: { type: Boolean, default: false }, // True only after Admin approves ID
  idCardImage: { type: String, default: "" }, // URL to stored ID image
  verificationStatus: {
    type: String,
    enum: ["UNVERIFIED", "PENDING", "VERIFIED", "REJECTED"],
    default: "UNVERIFIED"
  },

  // Financials (Internal Ledger)
  walletBalance: { type: Number, default: 0.0 },

  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model("User", userSchema);
```

3.2 Product Entity (models/Product.js)

Implements the "Strict Wizard" logic. Seller cannot type random text for core fields.

```
const mongoose = require("mongoose");

const productSchema = mongoose.Schema({
  // Relationship
  seller: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },

  // Strict Categorization (Dropdowns in UI)
  category: {
    type: String,
    required: true,
    enum: ["Mobile", "Tablet", "Laptop", "Desktop"]
  },
  brand: { type: String, required: true }, // e.g., "Apple"
  model: { type: String, required: true }, // e.g., "iPhone 13 Pro"

  // Technical Specifications
  specs: {
    storage: { type: String }, // e.g., "256GB"
    ram: { type: String },     // e.g., "16GB" (Crucial for Laptops)
    processor: { type: String }, // e.g., "M1", "Intel i7"
    batteryHealth: { type: Number } // e.g., 92 (Only for phones)
  },

  // Descriptive Condition Grading
  condition: {
    type: String,
    required: true,
    enum: ["Brand New", "Like New", "Good", "Fair", "For Parts"]
  },

  description: { type: String, required: true, maxlength: 1000 },
  price: { type: Number, required: true, min: 0 },
  images: [{ type: String, required: true }], // Array of Image URLs (Max 10)

  // Lifecycle
  status: {
    type: String,
    default: "AVAILABLE",
    enum: ["AVAILABLE", "PENDING", "SOLD", "HIDDEN"]
  },

  // Location Snapshot (In case seller moves)
  location: { type: String, required: true }, // Sub-city at time of listing

  createdAt: { type: Date, default: Date.now }
});

// Index for Search Performance
productSchema.index({ brand: 'text', model: 'text', description: 'text' });

module.exports = mongoose.model("Product", productSchema);
```

3.3 Order Entity (models/Order.js)

This controls the **Escrow Workflow**. It is the state machine for the transaction.

```
const mongoose = require("mongoose");

const orderSchema = mongoose.Schema({
  // Participants
  buyer: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  seller: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  product: { type: mongoose.Schema.Types.ObjectId, ref: "Product", required: true },

  // Financial Snapshot
  totalAmount: { type: Number, required: true }, // Price frozen at time of order
  appFee: { type: Number, default: 0 }, // For future monetization

  // The Escrow State Machine (Critical Logic)
  status: {
    type: String,
    default: "PENDING_PAYMENT",
    enum: [
      "PENDING_PAYMENT", // 1. Order created, waiting for payment
      "ESCROW_HELD",     // 2. Paid. Money is safe in App Wallet.
      "SHIPPED",         // 3. Seller marked as sent
      "DELIVERED",       // 4. Buyer confirmed receipt (or QR scan)
      "COMPLETED",       // 5. 24hrs passed OR Buyer accepted. Money released.
      "DISPUTED",        // 6. Buyer reported a problem. Admin needed.
      "CANCELLED"        // 7. Cancelled before payment
    ]
  },

  // Logistics
  deliveryMethod: { type: String, enum: ["Meetup", "Shipping"], required: true },
  trackingInfo: { type: String, default: "" }, // Tracking number or meeting place

  // Escrow Timers
  paidAt: { type: Date },
  shippedAt: { type: Date },
  deliveredAt: { type: Date }, // The moment the "24hr Clock" starts
  completedAt: { type: Date },

  createdAt: { type: Date, default: Date.now },
});

module.exports = mongoose.model("Order", orderSchema);
```

3.4 Transaction Entity (models/Transaction.js)

An immutable record of money movement. Required for trust and debugging.

```
const mongoose = require("mongoose");

const transactionSchema = mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  orderId: { type: mongoose.Schema.Types.ObjectId, ref: "Order" }, // Optional (if it's a deposit)

  amount: { type: Number, required: true },
  type: {
    type: String,
    enum: ["DEPOSIT", "WITHDRAWAL", "PURCHASE_HOLD", "SALE_EARNING", "REFUND"],
    required: true
  },

  description: { type: String }, // e.g., "Funds released for iPhone 13"
  status: { type: String, enum: ["PENDING", "SUCCESS", "FAILED"], default: "PENDING" },

  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model("Transaction", transactionSchema);
```

Part 2: API Specification & Endpoint Architecture

1. General API Standards

- **Base URL:** http://localhost:3000/api/v1
- **Data Format:** JSON (JavaScript Object Notation).
- **Authentication:** Bearer Token (JWT) in the Authorization header.
    - *Header Example:* Authorization: Bearer <your_jwt_token>
- **Response Structure:**

```
JSON

{
  "success": true,
  "message": "Operation successful",
  "data": { ... }
}
```

2. Authentication Module (routes/auth.js)

These routes handle user entry and security.

Method	Endpoint	Access	Description
POST	`/auth/signup`	Public	Register new user. Request body: `{ name, email, password, phone }`
POST	`/auth/login`	Public	Login. Returns `token` and `user` object.
POST	`/auth/upload-id`	**Private**	Upload Government ID image. Updates `verificationStatus` to `PENDING`.
GET	`/auth/me`	**Private**	Get current user's profile and wallet balance.

3. User & Admin Module (routes/user.js)

For managing profiles and admin approvals.

Method	Endpoint	Access	Description
PUT	`/users/profile`	**Private**	Update address or profile picture.
GET	`/users/admin/pending-ids`	**Admin**	**[Admin]** Get list of users waiting for ID verification.
PUT	`/users/admin/verify-user`	**Admin**	**[Admin]** Approve or Reject a user's ID. Body: `{ userId, status: 'VERIFIED' }`

4. Product Module (routes/product.js)

Implements the "Strict Listing Wizard" and Search.

Method	Endpoint	Access	Description
POST	`/products/add`	**Verified**	Create a new listing. Body must match the *Strict Schema* (Brand, Model, Condition).
GET	`/products`	Public	Get all products with query params for filtering. *Ex:* `/products?category=Mobile&minPrice=5000`
GET	`/products/search/:key`	Public	Text search for products.
GET	`/products/:id`	Public	Get details of a single product.
DELETE	`/products/:id`	**Owner**	Delete a listing (only if not currently in an active Order).

5. Order & Escrow Module (routes/order.js)

The "State Machine" logic. This is the most critical section.

Method	Endpoint	Access	Description
POST	`/orders/create`	**Verified**	**[Step 1]** Buyer clicks "Buy". Body: `{ productId, deliveryMethod }`. *Logic:* Checks Wallet Balance → Deducts Money → Sets Status `ESCROW_HELD`.
GET	`/orders/my-orders`	**Private**	Get history of items I bought.
GET	`/orders/my-sales`	**Private**	Get history of items I sold.
PUT	`/orders/:id/status`	**Private**	**[Step 2 & 3]** Update Order Status. *Seller Logic:* Send `{ status: 'SHIPPED' }`. *Buyer Logic:* Send `{ status: 'DELIVERED' }` (Start Timer) or `{ status: 'COMPLETED' }` (Release Money).
POST	`/orders/:id/dispute`	**Private**	Buyer reports issue. Sets Status `DISPUTED`. Alerts Admin.

6. Wallet & Transaction Module (routes/wallet.js)

Managing money.

Method	Endpoint	Access	Description
GET	`/wallet/history`	Private	View all transactions (Credits/Debits).
POST	`/wallet/deposit`	Private	Mock endpoint to add fake money (for testing).
POST	`/wallet/withdraw`	Private	Request payout. Body: `{ amount, bankDetails }`.

3. Recommended Folder Structure

A. Backend Structure (/server)

```
server/
├── config/
│   └── db.js           # MongoDB Connection logic
├── controllers/        # The logic functions (e.g., authController.js)
├── middlewares/
│   ├── auth.js         # Checks if JWT is valid
│   ├── admin.js        # Checks if User is Admin
│   └── upload.js       # Multer config for images
├── models/             # (You already have these files from SDD Part 1)
├── routes/             # (The API Endpoints listed above)
├── utils/              # Helper functions (e.g., error handling)
├── index.js            # Main App Entry
└── .env                # Secret keys (MONGO_URI, JWT_SECRET)
```

B. Frontend Structure (/lib in Flutter)

```
lib/
├── common/              # Reusable widgets (CustomButton, CustomTextField)
├── constants/           # Global variables (GlobalVariables.primaryColor)
├── features/            # Feature-based architecture
│   ├── auth/            # Screens: Login, Signup, OTP
│   ├── home/            # Screens: HomeScreen, Search
│   ├── product_details/ # Screens: ProductDetail, SellerProfile
│   ├── listing/         # Screens: AddProduct (The Wizard)
│   ├── cart/            # Screens: Cart, Checkout
│   ├── account/         # Screens: Profile, Wallet, OrderHistory
│   └── admin/           # Screens: Dashboard (for Admin users)
├── models/              # Dart classes matching Mongoose schemas
├── providers/           # State Management (UserProvider, ProductProvider)
└── router.router.dart   # Navigation configuration
```